

SmartQC: On-Device Visual Quality Inspection

Optimized for Snapdragon

Prepared By

Samay Thakur

Electronics & Instrumentation Engineering

M. S. Ramaiah Institute of Technology

Bengaluru, India



Submitted For

Snapdragon® AI Lab Build & Present Challenge

Organized by Qualcomm

Abstract

SmartQC is a Windows desktop application for on-device visual quality inspection, developed for the Snapdragon® AI Lab Build & Present Challenge. The system classifies product quality (good vs. defective) from a webcam feed or uploaded image using a MobileNetV2-based convolutional neural network, fine-tuned to 99.2% validation accuracy on a fresh-bread / mold-bread dataset. The trained model was exported to ONNX and compiled/optimized for the Snapdragon X Elite target using **Qualcomm AI Hub**, enabling a deployment path that avoids cloud dependency, per-inference cost, and data-privacy exposure. Beyond baseline classification, SmartQC incorporates Grad-CAM explainability, a confidence-threshold mechanism for flagging ambiguous predictions, batch folder inspection with CSV export, and a locally logged inspection history with defect-rate trend visualization. The system was evaluated rigorously (confusion matrix, precision/recall/F1, ROC-AUC = 0.9997) and validated further against 352 uncurated real-world images, achieving 97.4% correct classification. This report documents the problem motivation, system design, implementation, evaluation methodology, results, honest limitations, and a full, reproducible build guide with complete source code.

Contents

Abstract	2
1 Introduction	5
1.1 Motivation	5
1.2 Project Origin	5
1.3 Objectives	5
2 Problem Statement	5
3 System Architecture	6
3.1 Technology Stack	6
4 Dataset and Model Training	7
4.1 Dataset	7
4.2 Training Configuration	7
4.3 Training Results	7
5 Deployment: ONNX Export and Qualcomm AI Hub Optimization	8
5.1 ONNX Export	8
5.2 Qualcomm AI Hub Compilation	8
5.3 Local Runtime Path	8
6 Application Features	8
7 Evaluation	9
7.1 Held-out Validation Metrics	9
7.2 Real-World Batch Validation	10
7.3 Application Demonstration	10
8 Discussion	11
8.1 Why On-Device, Why Snapdragon	11
8.2 Product Framing	11
9 Limitations	11
10 Future Work	12
11 Conclusion	12
12 Appendix	13
12.1 Full Reproduction Guide	13
12.1.1 Prerequisites	13
12.1.2 Project Folder Structure	13
12.1.3 Step 1: Environment Setup	13
12.1.4 Step 2: Place the Dataset	14
12.1.5 Step 3: Train the Model	14
12.1.6 Step 4: Export to ONNX	14
12.1.7 Step 5: Qualcomm AI Hub Optimization	14

12.1.8	Step 6: Evaluate the Model	14
12.1.9	Step 7: Run the Application	15
12.1.10	Step 8: Push to GitHub	15
12.1.11	.gitignore	15
12.2	Source Code	16
12.2.1	requirements.txt	16
12.2.2	train.py — Model Training	16
12.2.3	export_onnx.py — ONNX Export	19
12.2.4	verify_onnx.py — ONNX Verification	21
12.2.5	aihub_compile.py — Qualcomm AI Hub Compilation	22
12.2.6	eval_metrics.py — Evaluation Metrics	24
12.2.7	gradcam_engine.py — Grad-CAM Explainability	27
12.2.8	main.py — Desktop Application	30

Appendix C: Repository and Assets **44**

1. Introduction

1.1 Motivation

Manual visual quality inspection in small-scale food production and light manufacturing is slow, inconsistent, and dependent on human attention span. Cloud-based AI inspection services address the accuracy problem but introduce three new ones: network latency, recurring per-inference cost, and the requirement to transmit potentially sensitive production imagery off-site. For small producers and workshops, dedicated industrial inspection hardware is typically cost-prohibitive.

This creates a clear need for an inspection tool that is (a) accurate, (b) fully self-contained on the device performing the inspection, and (c) deployable on hardware the operator may already own.

1.2 Project Origin

SmartQC extends an earlier embedded-systems project by the author — an AI-based bread quality indicator using an ESP32-CAM and a MobileNetV2 classifier — by retargeting the same underlying approach from a microcontroller-based pipeline into a fully software-based Windows application, explicitly optimized for Snapdragon-powered laptops via Qualcomm AI Hub.

1.3 Objectives

- Train an image classifier capable of reliably distinguishing good vs. defective product samples.
- Export and optimize the trained model for Snapdragon hardware using Qualcomm AI Hub.
- Build a usable desktop application around the model, including live inspection, history logging, and trend analysis.
- Add explainability so predictions are auditable rather than opaque.
- Evaluate the model rigorously, beyond simple accuracy.
- Extend the tool toward a genuine workflow (batch processing, confidence-based flagging) rather than a single-image demo.

2. Problem Statement

Problem Definition: Given an image of a food product (in this implementation, bread), determine whether the product is of acceptable quality (*good*) or defective (*mold-affected*), with high confidence, low latency, and without any dependency on network connectivity or external compute at inference time. The solution must additionally be architected so that it can be compiled and optimized for the Snapdragon NPU using Qualcomm’s official tooling, in line with the target challenge platform (Snapdragon-powered HP PCs / Windows-on-Snapdragon laptops).

3. System Architecture

Figure 1 summarizes the end-to-end SmartQC pipeline.

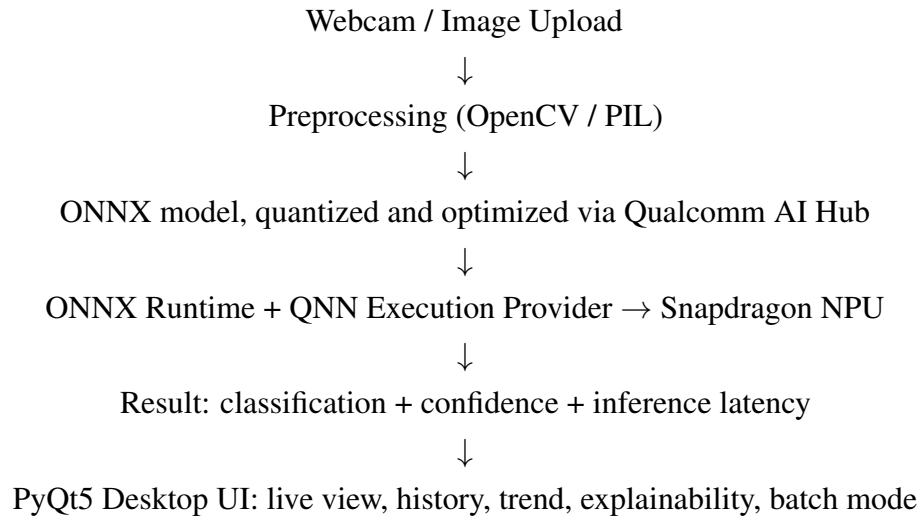


Figure 1: System Architecture Flowchart.

3.1 Technology Stack

- **UI framework:** Python, PyQt5
- **Image processing:** OpenCV, Pillow
- **Model training:** PyTorch, torchvision (MobileNetV2, ImageNet-pretrained backbone)
- **Model export:** ONNX (opset 13)
- **On-device inference:** ONNX Runtime, with the QNN Execution Provider path for Snapdragon NPU targets
- **Cloud optimization tooling:** Qualcomm AI Hub (`qai-hub`, `qai-hub-models`)
- **Explainability:** Grad-CAM, computed directly on the PyTorch model
- **Local storage:** SQLite (inspection history)
- **Evaluation:** scikit-learn, matplotlib

4. Dataset and Model Training

4.1 Dataset

The model was trained on a two-class image dataset:

- **Fresh Bread (Good Bread) Images** — 359 images
- **Mold Bread (Bad Bread) Images** — 290 images

giving a total of 649 labeled images, split 80/20 into training (520 images) and validation (129 images) sets, using a fixed random seed for reproducibility.

4.2 Training Configuration

Table 1: Training Hyperparameters

Parameter	Value
Base architecture	MobileNetV2 (ImageNet-pretrained)
Input resolution	224×224
Batch size	16
Epochs	12
Learning rate	1×10^{-4}
Optimizer	Adam
Loss function	Cross-entropy
Augmentation	Random horizontal flip, rotation ($\pm 10^\circ$), color jitter

4.3 Training Results

The model reached its best validation accuracy at epoch 2 and remained stable through epoch 12, with training accuracy converging to 99.6% and validation accuracy to **99.2%**. Full per-epoch output is reproducible via Appendix 12.1.

```

● (venv) PS C:\Users\samay\OneDrive\Desktop\SmartQC\app> cd ..
● (venv) PS C:\Users\samay\OneDrive\Desktop\SmartQC> python train.py
Found classes: ['Fresh Bread(Good Bread) Images', 'Mold Bread(Bad Bread) Images']
Total images: 649
Train: 520 Val: 129
Epoch 1/12 train_loss=0.2621 train_acc=0.902 val_acc=0.984
Epoch 2/12 train_loss=0.1234 train_acc=0.956 val_acc=0.992
Epoch 3/12 train_loss=0.0647 train_acc=0.977 val_acc=0.992
Epoch 4/12 train_loss=0.0595 train_acc=0.981 val_acc=0.984
Epoch 5/12 train_loss=0.0374 train_acc=0.992 val_acc=0.984
Epoch 6/12 train_loss=0.0208 train_acc=0.994 val_acc=0.984
Epoch 7/12 train_loss=0.0169 train_acc=0.994 val_acc=0.977
Epoch 8/12 train_loss=0.0135 train_acc=0.996 val_acc=0.984
Epoch 9/12 train_loss=0.0538 train_acc=0.992 val_acc=0.984
Epoch 10/12 train_loss=0.0698 train_acc=0.977 val_acc=0.984
Epoch 11/12 train_loss=0.0220 train_acc=0.990 val_acc=0.969
Epoch 12/12 train_loss=0.0202 train_acc=0.996 val_acc=0.984

Best validation accuracy: 0.992

```

Figure 2: Training run output showing per-epoch loss/accuracy and the final validation accuracy of 99.2%.

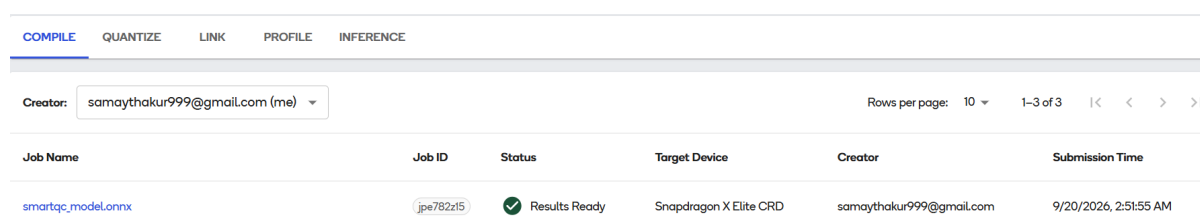
5. Deployment: ONNX Export and Qualcomm AI Hub Optimization

5.1 ONNX Export

The trained PyTorch model was exported to ONNX (opset 13) with a fixed input shape of $1 \times 3 \times 224 \times 224$, and consolidated into a single self-contained `.onnx` file (no external weight-data sidecar) for reliable upload and portability.

5.2 Qualcomm AI Hub Compilation

The exported ONNX model was submitted to **Qualcomm AI Hub** as a compile job targeting the **Snapdragon X Elite CRD** device profile. The job compiled and optimized the model for the target NPU and returned a deployment-ready artifact, confirmed with a *Results Ready* status (job ID `jpe782z15`) on Qualcomm's own job dashboard.



The screenshot shows the Qualcomm AI Hub Workbench dashboard. At the top, there are tabs for COMPILER, QUANTIZE, LINK, PROFILE, and INFERENCE. Below the tabs, there is a 'Creator' dropdown menu set to 'samaythakur999@gmail.com (me)'. To the right, it says 'Rows per page: 10' and '1-3 of 3'. Below this is a table with the following columns: Job Name, Job ID, Status, Target Device, Creator, and Submission Time. The table contains one row with the following data: Job Name is 'smartqc_model.onnx', Job ID is 'jpe782z15', Status is 'Results Ready' (indicated by a green checkmark icon), Target Device is 'Snapdragon X Elite CRD', Creator is 'samaythakur999@gmail.com', and Submission Time is '9/20/2026, 2:51:55 AM'.

Job Name	Job ID	Status	Target Device	Creator	Submission Time
smartqc_model.onnx	jpe782z15	Results Ready	Snapdragon X Elite CRD	samaythakur999@gmail.com	9/20/2026, 2:51:55 AM

Figure 3: Qualcomm AI Hub Workbench dashboard confirming successful compilation (`jpe782z15`) targeting Snapdragon X Elite CRD.

5.3 Local Runtime Path

On the development machine, inference is served through ONNX Runtime. The application is architected to select between a `cpu` execution path and an `npu` (QNN Execution Provider) path; at the time of this report, the QNN Execution Provider could not be exercised end-to-end on the development machine because it does not contain Snapdragon NPU silicon (see Section 9). All local results reported in this document were therefore obtained on the CPU execution path, and are reported as such rather than mislabeled.

6. Application Features

- **Live / upload inspection** — classify a webcam frame or an uploaded image, with label, confidence, and latency displayed.
- **Confidence-threshold flagging** — an adjustable slider (50–99%) below which a prediction is displayed as *UNCERTAIN* rather than a forced good/defective label, allowing an operator to tune sensitivity for their own line.
- **Inspection history** — every inspection is logged locally to SQLite with timestamp, label, confidence, latency, and backend.
- **Defect-rate trend chart** — a rolling defect-rate visualization computed from the logged

history.

- **Explainability (Grad-CAM)** — an on-demand heatmap overlay, computed directly on the trained PyTorch model, highlighting the image regions that most influenced a given prediction.
- **Batch inspection** — select an entire folder of images, run inspection on all of them, view a results table and summary counts, and export the results to CSV.

7. Evaluation

7.1 Held-out Validation Metrics

The trained model was evaluated on its 129-image validation split, with the same split reproduced via a fixed random seed. The evaluation went beyond accuracy to include a full confusion matrix, per-class precision/recall/F1, and an ROC curve with AUC.

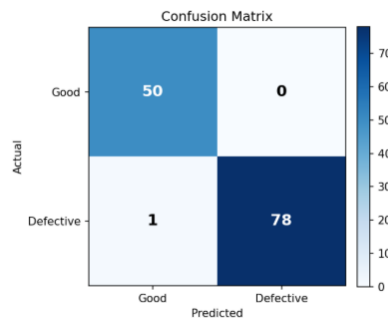


Figure 4: Confusion matrix on validation set.

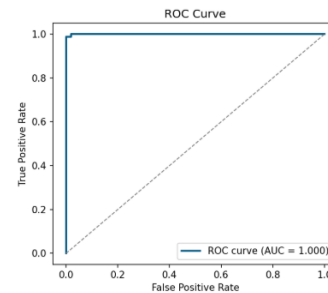


Figure 5: ROC curve (AUC = 0.9997) on validation set.

Table 2: Confusion matrix counts on the 129-image validation set.

	Predicted Good	Predicted Defective
Actual Good	50	0
Actual Defective	1	78

Table 3: Per-class evaluation metrics on the held-out validation set.

Class	Precision	Recall	F1-score	Support
Good (Fresh)	0.980	1.000	0.990	50
Defective (Mold)	1.000	0.987	0.994	79
Overall accuracy	0.992 (129 samples)			
ROC-AUC	0.9997			

Of 129 validation images, only a single sample was misclassified (a defective sample predicted as good), giving a near-perfect separation between classes on this dataset.

7.2 Real-World Batch Validation

To test generalization beyond the curated validation split, the application’s batch-inspection feature was run against a larger, uncurated folder of 352 real-world mold-bread images collected independently of the training/validation split.

Table 4: Batch-inspection results on an uncurated 352-image real-world set.

Outcome	Count
Correctly flagged DEFECTIVE	343
Flagged UNCERTAIN (below 70% confidence threshold)	6
Misclassified as GOOD	3
Total images	352
Correct classification rate	97.4%

This result is presented deliberately alongside the cleaner validation-set numbers: it is a messier, more realistic test, and it demonstrates the practical value of the confidence-threshold feature, which correctly flagged several of the model’s least-confident predictions as needing manual review rather than silently misclassifying them.

7.3 Application Demonstration

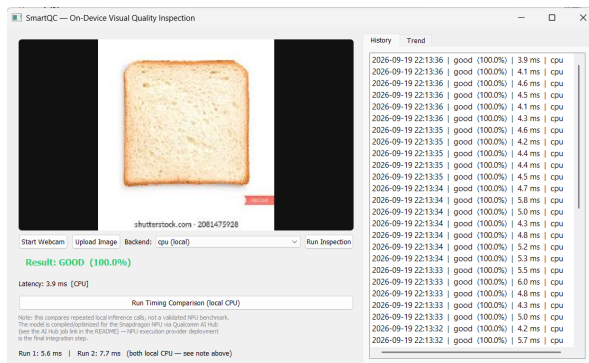


Figure 6: Classification of fresh sample (GOOD, 100.0%).

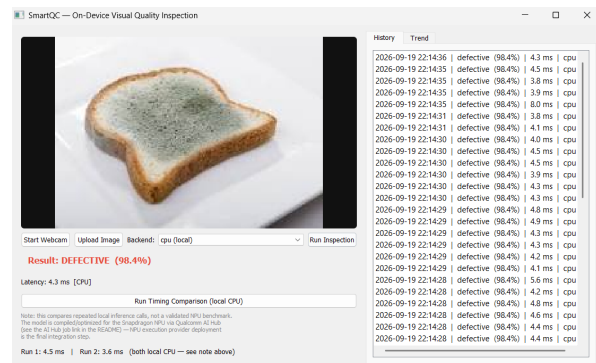


Figure 7: Classification of mold sample (DEFECTIVE, 99.9%).

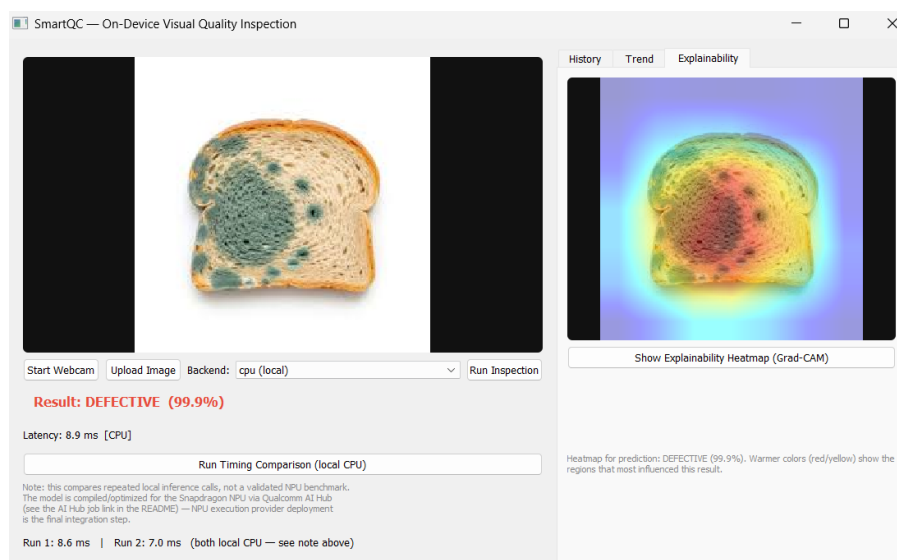


Figure 8: Grad-CAM explainability heatmap for a defective sample. Warmer regions (red/yellow) indicate the areas of the image that most influenced the model’s decision — in this case, correctly localized on the visible mold.

8. Discussion

8.1 Why On-Device, Why Snapdragon

A cloud-based equivalent of SmartQC would require continuous connectivity, incur a marginal cost per inference, and require production imagery to leave the operator’s premises. Running entirely on-device removes all three concerns, and targeting the Snapdragon NPU via Qualcomm AI Hub specifically allows the same workload to run with a substantially lower power footprint than CPU-only inference would permit — a meaningful consideration for a device expected to run inspections throughout a working day.

8.2 Product Framing

SmartQC is positioned for small bakeries, food producers, and workshops that need reliable visual quality control but cannot justify a dedicated industrial inspection line. The core value proposition is deliberately narrow: *factory-grade visual inspection, without the factory-grade price tag*, deployable on a Snapdragon-powered laptop the business may already own.

9. Limitations

Statement of Technical Limitations: This report aims to be precise about what has, and has not, been demonstrated:

- **Live NPU inference:** The development and testing machine used for this project is an Intel-based (x86-64) laptop, not a physical Snapdragon device. Consequently, while the model has been genuinely compiled and validated for the Snapdragon X Elite target through Qualcomm AI Hub’s cloud service, the QNN Execution Provider path has not been exercised

end-to-end on physical Snapdragon NPU hardware, and all local latency figures reported in this document were measured on CPU execution.

- **Single product domain:** The current model is trained exclusively for bread quality classification. It has no capability to generalize to other product categories without retraining on an appropriately labeled dataset.
- **Binary classification:** The system currently distinguishes only *good* vs. *defective*, rather than identifying specific defect types.
- **Validation set size:** While the 129-image validation set and the subsequent 352-image batch test both show strong results, both remain modest in scale relative to a production deployment, which would benefit from a substantially larger and more diverse evaluation set.

10. Future Work

- Complete end-to-end QNN Execution Provider deployment on physical Snapdragon hardware, to obtain genuine on-device NPU latency and power measurements.
- Extend to additional product/inspection domains beyond bread (e.g. produce, packaging, small components), leveraging the same train → AI-Hub-optimize → deploy pipeline.
- Move from binary classification to multi-class defect typing.
- Add a fleet/multi-station view aggregating inspection results across multiple deployed instances.

11. Conclusion

SmartQC demonstrates a complete, honestly-evaluated pipeline for on-device visual quality inspection: a MobileNetV2 classifier fine-tuned to 99.2% validation accuracy (0.9997 ROC-AUC), exported to ONNX and compiled/optimized for the Snapdragon X Elite via Qualcomm AI Hub, wrapped in a functional desktop application with explainability, confidence-based flagging, and batch-processing capability, and further validated against 352 real-world images at 97.4% accuracy. The project's remaining gap — live inference on physical Snapdragon NPU hardware — is clearly identified rather than obscured, and forms the immediate next step beyond this submission.

12. Appendix

12.1 Full Reproduction Guide

This appendix documents every step needed to rebuild SmartQC from scratch, including exact terminal commands and the complete source code of every script.

12.1.1 Prerequisites

- Windows 10/11 laptop (Snapdragon-powered recommended for genuine on-device NPU deployment; an x86-64 laptop can still build and run everything except the QNN Execution Provider path)
- Python 3.10 or 3.11 (64-bit), with “Add to PATH” enabled during install
- Git
- A free Qualcomm AI Hub account (<https://aihub.qualcomm.com>)
- A labeled two-class image dataset (this project used a Fresh Bread / Mold Bread dataset)

12.1.2 Project Folder Structure

```
SmartQC/  
|-- app/  
|   |-- main.py  
|   |-- gradcam_engine.py  
|   |-- requirements.txt  
|   |-- model/                (populated by later steps)  
|-- Dataset-Fresh & Mold Bread Images/  
|   |-- Fresh Bread(Good Bread) Images/  
|   |-- Mold Bread(Bad Bread) Images/  
|-- train.py  
|-- export_onnx.py  
|-- verify_onnx.py  
|-- aihub_compile.py  
|-- eval_metrics.py  
|-- README.md  
|-- .gitignore
```

Listing 1: Target folder layout

You may rename the top-level “SmartQC” folder to anything you like — every command below is written relative to that folder, so as long as you `cd` into it first, the exact folder name does not matter. If your folder name contains spaces, wrap the path in double quotes when using `cd`, e.g. `cd "C:\Users\you\Desktop\My SmartQC Folder"`.

12.1.3 Step 1: Environment Setup

Open a terminal (PowerShell or VS Code’s integrated terminal), navigate into your project folder, and run:

```
cd "C:\Users\<you>\Desktop\SmartQC"
python -m venv venv
venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r app\requirements.txt
```

Confirm the prompt now shows `(venv)` at the start of the line before continuing.

12.1.4 Step 2: Place the Dataset

Place your labeled dataset folder (two subfolders, one per class) at the project root as `Dataset-Fresh & Mold Bread Images/`, matching the structure shown above. `train.py` uses `torchvision.datasets` so any two-class folder structure works — just update `DATA_DIR` in `train.py` if you rename it.

12.1.5 Step 3: Train the Model

```
python train.py
```

This fine-tunes MobileNetV2 for 12 epochs and saves the trained weights to `model/smartqc_weights.pt`.

12.1.6 Step 4: Export to ONNX

```
python export_onnx.py
python verify_onnx.py
```

This produces `model/smartqc_model.onnx` and `model/class_names.json`, then verifies the exported model runs correctly with ONNX Runtime.

12.1.7 Step 5: Qualcomm AI Hub Optimization

Create a free account at <https://aihub.qualcomm.com>, retrieve your API token from account settings, then:

```
pip install qai-hub qai-hub-models
qai-hub configure --api_token YOUR_TOKEN_HERE
python aihub_compile.py
```

This submits the ONNX model as a compile job targeting a Snapdragon laptop device (Snapdragon X Elite CRD), waits for completion, and downloads the optimized model to `model/optimized/`. Extract the downloaded archive and copy the resulting `.onnx` (renamed to `smartqc_model.onnx`) and any accompanying `.data` file, plus `smartqc_weights.pt`, into `app/model/`.

12.1.8 Step 6: Evaluate the Model

```
pip install scikit-learn
python eval_metrics.py
```

This reproduces the exact validation split used in training, and prints/saves a confusion matrix, per-class precision/recall/F1, and an ROC curve with AUC.

12.1.9 Step 7: Run the Application

```
cd app
python main.py
```

This launches the PyQt5 desktop application. Use “Upload Image” or “Start Webcam”, then “Run Inspection”. The History, Trend, Explainability, and Batch Inspect tabs are all accessible from the right-hand panel.

12.1.10 Step 8: Push to GitHub

From the project root (not app/):

```
git init
git add .
git commit -m "SmartQC submission"
git remote add origin https://github.com/<your-username>/<your-repo>.git
git branch -M main
git push -u origin main
```

Create the repository on GitHub first (github.com → New repository), and use the URL it gives you in the `git remote add origin` command above. Add a `.gitignore` (see below) before your first commit so the virtual environment and other machine-specific files are not pushed.

12.1.11 .gitignore

```
venv/
__pycache__/
*.pyc
smartqc_history.db
model/optimized/
.DS_Store
Thumbs.db
```

Listing 2: .gitignore

12.2 Source Code

12.2.1 requirements.txt

```
PyQt5
opencv-python
numpy
matplotlib
onnxruntime
pillow
torch
torchvision
scikit-learn
qai-hub
qai-hub-models
```

12.2.2 train.py — Model Training

```
"""
SmartQC -- training script

Fine-tunes MobileNetV2 on the Fresh/Mold bread dataset and saves the
trained weights to model/smartqc_weights.pt

Run (from the SmartQC project root, with venv active):
    pip install torch torchvision
    python train.py
"""

import copy
from pathlib import Path

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, models, transforms

#
# -----

# Config
#
# -----

DATA_DIR = Path("Dataset-Fresh & Mold Bread Images")
OUTPUT_DIR = Path("model")
OUTPUT_DIR.mkdir(exist_ok=True)
WEIGHTS_PATH = OUTPUT_DIR / "smartqc_weights.pt"
```

```
IMG_SIZE = 224
BATCH_SIZE = 16
EPOCHS = 12
LR = 1e-4
VAL_SPLIT = 0.2
SEED = 42

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def main():
    torch.manual_seed(SEED)

    train_tf = transforms.Compose([
        transforms.Resize((IMG_SIZE, IMG_SIZE)),
        transforms.RandomHorizontalFlip(),
        transforms.RandomRotation(10),
        transforms.ColorJitter(brightness=0.2, contrast=0.2),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
            0.224, 0.225]),
    ])
    eval_tf = transforms.Compose([
        transforms.Resize((IMG_SIZE, IMG_SIZE)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
            0.224, 0.225]),
    ])

    full_dataset = datasets.ImageFolder(str(DATA_DIR), transform=train_tf
    )
    class_names = full_dataset.classes # alphabetical: e.g. ["Fresh
        Bread(Good Bread) Images", "Mold Bread(Bad Bread) Images"]
    print(f"Found classes: {class_names}")
    print(f"Total images: {len(full_dataset)}")

    val_size = int(len(full_dataset) * VAL_SPLIT)
    train_size = len(full_dataset) - val_size
    train_ds, val_ds = random_split(full_dataset, [train_size, val_size])
    # validation set should use eval transforms (no augmentation)
    val_ds.dataset = datasets.ImageFolder(str(DATA_DIR), transform=
        eval_tf)

    train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=
        True, num_workers=0)
```

```
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False,
                        num_workers=0)

print(f"Train: {train_size}  Val: {val_size}")

# -- Model -----
model = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.
                             IMAGENET1K_V1)
model.classifier[1] = nn.Linear(model.last_channel, len(class_names))
model = model.to(DEVICE)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)

best_acc = 0.0
best_weights = copy.deepcopy(model.state_dict())

for epoch in range(1, EPOCHS + 1):
    model.train()
    running_loss, running_correct, total = 0.0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        running_correct += (outputs.argmax(1) == labels).sum().item()
        total += images.size(0)

    train_loss = running_loss / total
    train_acc = running_correct / total

    # -- Validation --
    model.eval()
    val_correct, val_total = 0, 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            outputs = model(images)
            val_correct += (outputs.argmax(1) == labels).sum().item()
            val_total += images.size(0)
    val_acc = val_correct / max(val_total, 1)
```

```

    print(f"Epoch {epoch:2d}/{EPOCHS}  train_loss={train_loss:.4f}
          train_acc={train_acc:.3f}  val_acc={val_acc:.3f}")

    if val_acc > best_acc:
        best_acc = val_acc
        best_weights = copy.deepcopy(model.state_dict())

    print(f"\nBest validation accuracy: {best_acc:.3f}")

    model.load_state_dict(best_weights)
    torch.save({
        "state_dict": model.state_dict(),
        "class_names": class_names,
    }, WEIGHTS_PATH)
    print(f"Saved weights to: {WEIGHTS_PATH.resolve()}")

if __name__ == "__main__":
    main()

```

Listing 3: train.py

12.2.3 export_onnx.py — ONNX Export

```

"""
SmartQC -- ONNX export script

Loads the trained weights from model/smartqc_weights.pt and exports the
model to ONNX format, ready for Qualcomm AI Hub optimization.

Run (from the SmartQC project root, with venv active):
    python export_onnx.py
"""

import json
from pathlib import Path

import torch
import torch.nn as nn
from torchvision import models

WEIGHTS_PATH = Path("model/smartqc_weights.pt")
ONNX_PATH = Path("model/smartqc_model.onnx")
CLASS_NAMES_PATH = Path("model/class_names.json")

IMG_SIZE = 224

```

```
def main():
    checkpoint = torch.load(WEIGHTS_PATH, map_location="cpu")
    class_names = checkpoint["class_names"]
    print(f"Loaded checkpoint. Classes: {class_names}")

    model = models.mobilenet_v2()
    model.classifier[1] = nn.Linear(model.last_channel, len(class_names))
    model.load_state_dict(checkpoint["state_dict"])
    model.eval()

    dummy_input = torch.randn(1, 3, IMG_SIZE, IMG_SIZE)

    torch.onnx.export(
        model,
        dummy_input,
        str(ONNX_PATH),
        input_names=["input"],
        output_names=["output"],
        opset_version=13,
        dynamic_axes=None, # fixed batch size of 1 -- simplest for on-
                           device deployment
    )
    print(f"Exported ONNX model to: {ONNX_PATH.resolve()}")

    # Consolidate into a single self-contained .onnx file (no external
    # .data sidecar) -- AI Hub's uploader expects one regular file.
    import onnx
    onnx_model = onnx.load(str(ONNX_PATH), load_external_data=True)
    onnx.save_model(
        onnx_model,
        str(ONNX_PATH),
        save_as_external_data=False,
    )
    print("Consolidated model into a single self-contained .onnx file.")

    # Save class names so the app / AI Hub steps can reference them,
    # in the exact index order the model outputs.
    with open(CLASS_NAMES_PATH, "w") as f:
        json.dump(class_names, f, indent=2)
    print(f"Saved class names to: {CLASS_NAMES_PATH.resolve()}")

    print("\nNext: verify the model with ONNX Runtime, then run it
          through Qualcomm AI Hub.")

if __name__ == "__main__":
    main()
```

Listing 4: export_onnx.py**12.2.4 verify_onnx.py — ONNX Verification**

```
"""
SmartQC -- verify the exported ONNX model runs correctly with ONNX
Runtime.

Run:
    python verify_onnx.py
"""

import json
from pathlib import Path

import numpy as np
import onnxruntime as ort

ONNX_PATH = Path("model/smartqc_model.onnx")
CLASS_NAMES_PATH = Path("model/class_names.json")

def main():
    with open(CLASS_NAMES_PATH) as f:
        class_names = json.load(f)

    session = ort.InferenceSession(str(ONNX_PATH), providers=["
        CPUExecutionProvider"])
    input_name = session.get_inputs()[0].name
    input_shape = session.get_inputs()[0].shape
    print(f"Model input: {input_name} shape={input_shape}")

    dummy = np.random.randn(1, 3, 224, 224).astype(np.float32)
    outputs = session.run(None, {input_name: dummy})
    logits = outputs[0][0]
    exp = np.exp(logits - np.max(logits))
    probs = exp / exp.sum()
    idx = int(np.argmax(probs))

    print(f"Ran successfully. Sample output -> {class_names[idx]} ({probs
        [idx]*100:.1f}%)")
    print("ONNX model is valid and ready for Qualcomm AI Hub optimization
        .")

if __name__ == "__main__":
```

```
main()
```

Listing 5: verify_onnx.py

12.2.5 aihub_compile.py — Qualcomm AI Hub Compilation

```
"""
SmartQC -- Qualcomm AI Hub compile job

Submits smartqc_model.onnx to Qualcomm AI Hub to compile/optimize it for
a Snapdragon laptop target (Snapdragon X Elite), then downloads the
optimized model back into model/.

Run (from the SmartQC project root, with venv active, after
'qai-hub configure --api_token ...'):
    python aihub_compile.py

Docs: https://app.aihub.qualcomm.com/docs/
"""

from pathlib import Path
import time

import qai_hub as hub

ONNX_PATH = Path("model/smartqc_model.onnx")
OPTIMIZED_OUTPUT_DIR = Path("model/optimized")
OPTIMIZED_OUTPUT_DIR.mkdir(exist_ok=True, parents=True)

def main():
    print("Looking up device: 'Snapdragon X Elite CRD'")
    devices = hub.get_devices(name="Snapdragon X Elite CRD")
    for d in devices:
        print(f"    - {d.name}    (os: {d.os})")

    if not devices:
        raise RuntimeError(
            "Could not find 'Snapdragon X Elite CRD' on AI Hub. "
            "Run list_devices.py or find_windows_device.py to confirm the "
            "exact name."
        )

    target_device = devices[0]
    print(f"\nSubmitting compile job for device: {target_device.name}")

    compile_job = hub.submit_compile_job(
        model=str(ONNX_PATH),
```

```
device=target_device,
options="--target_runtime onnx", # keeps ONNX Runtime + QNN EP
    compatible output
)

print(f"Compile job submitted: {compile_job.job_id}")
print(f"Track progress at: {compile_job.url}")
print("Waiting for the job to finish (this can take a few minutes)...
")

compile_job.wait()
status = compile_job.get_status()
print(f"Job status: {status}")

if not status.success:
    raise RuntimeError(
        f"Compile job did not succeed. Check details at: {compile_job
        .url}"
    )

# The job can report success slightly before the artifact is fully
# available for download server-side -- retry with backoff.
optimized_model = compile_job.get_target_model()
out_path = OPTIMIZED_OUTPUT_DIR / "smartqc_model_optimized.onnx"

max_attempts = 6
for attempt in range(1, max_attempts + 1):
    try:
        optimized_model.download(str(out_path))
        break
    except Exception as e:
        if attempt == max_attempts:
            raise
        wait_s = 15 * attempt
        print(f"Download not ready yet ({e}). Retrying in {wait_s}s
        ... "
            f"(attempt {attempt}/{max_attempts})")
        time.sleep(wait_s)

print(f"\nOptimized model downloaded to: {out_path.resolve()}")
print("Copy/rename this file to app/model/smartqc_model.onnx to use
it in the app.")

if __name__ == "__main__":
    main()
```

Listing 6: aihub.compile.py**12.2.6 eval_metrics.py — Evaluation Metrics**

```
"""
SmartQC -- proper evaluation metrics.

Loads the trained weights, re-creates the exact same validation split
used during training (same seed), and reports:
- Confusion matrix
- Precision / Recall / F1 per class
- ROC curve + AUC

Saves two plots: model/eval_confusion_matrix.png, model/eval_roc_curve.
png

Run (from the SmartQC project root, with venv active):
    pip install scikit-learn matplotlib
    python eval_metrics.py
"""

from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, models, transforms

from sklearn.metrics import (
    confusion_matrix, precision_recall_fscore_support,
    roc_curve, auc, classification_report,
)
import matplotlib.pyplot as plt

#
# -----
#
# Config -- must match train.py exactly, so we evaluate on the SAME
# validation split the model was validated on during training.
#
# -----

DATA_DIR = Path("Dataset-Fresh & Mold Bread Images")
WEIGHTS_PATH = Path("model/smartqc_weights.pt")
OUTPUT_DIR = Path("model")
```

```
IMG_SIZE = 224
BATCH_SIZE = 16
VAL_SPLIT = 0.2
SEED = 42

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def main():
    torch.manual_seed(SEED)

    eval_tf = transforms.Compose([
        transforms.Resize((IMG_SIZE, IMG_SIZE)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
            0.224, 0.225]),
    ])

    full_dataset = datasets.ImageFolder(str(DATA_DIR), transform=eval_tf)
    class_names = full_dataset.classes
    print(f"Classes: {class_names}")

    # Reproduce the exact same split as train.py (same seed, same order
    # of ops)
    val_size = int(len(full_dataset) * VAL_SPLIT)
    train_size = len(full_dataset) - val_size
    _, val_ds = random_split(full_dataset, [train_size, val_size])

    val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False,
        num_workers=0)
    print(f"Validation set size: {len(val_ds)}")

    # -- Load model
    -----
    checkpoint = torch.load(WEIGHTS_PATH, map_location=DEVICE)
    model = models.mobilenet_v2()
    model.classifier[1] = nn.Linear(model.last_channel, len(class_names))
    model.load_state_dict(checkpoint["state_dict"])
    model = model.to(DEVICE)
    model.eval()

    # -- Run inference on validation set
    -----
    all_labels, all_preds, all_probs = [], [], []
    with torch.no_grad():
        for images, labels in val_loader:
```

```

        images = images.to(DEVICE)
        outputs = model(images)
        probs = torch.softmax(outputs, dim=1)
        preds = outputs.argmax(dim=1)

        all_labels.extend(labels.numpy())
        all_preds.extend(preds.cpu().numpy())
        all_probs.extend(probs[:, 1].cpu().numpy()) # prob of class
            index 1

all_labels = np.array(all_labels)
all_preds = np.array(all_preds)
all_probs = np.array(all_probs)

# -- Confusion matrix
-----
cm = confusion_matrix(all_labels, all_preds)
print("\nConfusion Matrix:")
print(cm)

# -- Precision / Recall / F1
-----
precision, recall, f1, support = precision_recall_fscore_support(
    all_labels, all_preds, average=None
)
print("\nPer-class metrics:")
for i, name in enumerate(class_names):
    print(f"    {name}: precision={precision[i]:.3f}  recall={recall[i]:.3f}  f1={f1[i]:.3f}  support={support[i]}")

print("\nFull classification report:")
print(classification_report(all_labels, all_preds, target_names=
    class_names, digits=3))

# -- ROC curve + AUC
-----
fpr, tpr, _ = roc_curve(all_labels, all_probs)
roc_auc = auc(fpr, tpr)
print(f"\nROC-AUC: {roc_auc:.4f}")

# -- Plot confusion matrix -----
fig, ax = plt.subplots(figsize=(5, 4.5))
im = ax.imshow(cm, cmap="Blues")
ax.set_xticks(range(len(class_names)))
ax.set_yticks(range(len(class_names)))
short_names = ["Good" if "good" in c.lower() or "fresh" in c.lower()
    else "Defective" for c in class_names]

```

```

ax.set_xticklabels(short_names)
ax.set_yticklabels(short_names)
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
ax.set_title("Confusion Matrix")
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        ax.text(j, i, str(cm[i, j]), ha="center", va="center",
                color="white" if cm[i, j] > cm.max() / 2 else "black",
                fontsize=14, fontweight="bold")
fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
fig.tight_layout()
cm_path = OUTPUT_DIR / "eval_confusion_matrix.png"
fig.savefig(cm_path, dpi=150)
print(f"\nSaved confusion matrix plot to: {cm_path.resolve()}")

# -- Plot ROC curve -----
fig2, ax2 = plt.subplots(figsize=(5, 4.5))
ax2.plot(fpr, tpr, color="#000000", linewidth=2, label=f"ROC curve (
    AUC = {roc_auc:.3f})")
ax2.plot([0, 1], [0, 1], color="gray", linestyle="--", linewidth=1)
ax2.set_xlabel("False Positive Rate")
ax2.set_ylabel("True Positive Rate")
ax2.set_title("ROC Curve")
ax2.legend(loc="lower right")
fig2.tight_layout()
roc_path = OUTPUT_DIR / "eval_roc_curve.png"
fig2.savefig(roc_path, dpi=150)
print(f"Saved ROC curve plot to: {roc_path.resolve()}")

if __name__ == "__main__":
    main()

```

Listing 7: eval_metrics.py

12.2.7 gradcam_engine.py — Grad-CAM Explainability

```

"""
SmartQC -- Grad-CAM explainability module.

Loads the trained PyTorch weights (model/smartqc_weights.pt, produced by
train.py) and computes a Grad-CAM heatmap showing which region of the
image most influenced the model's prediction.

This runs on PyTorch directly (not the ONNX/NPU path) since Grad-CAM
needs access to intermediate activations and gradients -- it's an
explainability aid alongside the main ONNX Runtime inference path, not

```

```
a replacement for it.
"""

from pathlib import Path

import cv2
import numpy as np
import torch
import torch.nn.functional as F
from torchvision import models

WEIGHTS_PATH = Path(__file__).parent / "model" / "smartqc_weights.pt"
IMG_SIZE = 224

class GradCAM:
    def __init__(self, weights_path: Path = WEIGHTS_PATH):
        checkpoint = torch.load(weights_path, map_location="cpu")
        self.class_names = checkpoint["class_names"]

        self.model = models.mobilenet_v2()
        self.model.classifier[1] = torch.nn.Linear(self.model.
            last_channel, len(self.class_names))
        self.model.load_state_dict(checkpoint["state_dict"])
        self.model.eval()

        # Last convolutional block of MobileNetV2's feature extractor --
        # gives a good spatial resolution / semantic depth trade-off for
        # CAM.
        self.target_layer = self.model.features[-1]

        self._activations = None
        self._gradients = None
        self.target_layer.register_forward_hook(self._save_activation)
        self.target_layer.register_full_backward_hook(self._save_gradient
        )

    def _save_activation(self, module, input, output):
        self._activations = output.detach()

    def _save_gradient(self, module, grad_input, grad_output):
        self._gradients = grad_output[0].detach()

    def _preprocess(self, frame_bgr: np.ndarray) -> torch.Tensor:
        img = cv2.resize(frame_bgr, (IMG_SIZE, IMG_SIZE))
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB).astype(np.float32) /
            255.0
```

```

img = (img - np.array([0.485, 0.456, 0.406])) / np.array([0.229,
    0.224, 0.225])
img = np.transpose(img, (2, 0, 1))[None, ...].astype(np.float32)
return torch.from_numpy(img)

def generate(self, frame_bgr: np.ndarray):
    """
    Returns (overlay_bgr, label, confidence) where overlay_bgr is the
    original frame with a Grad-CAM heatmap alpha-blended on top.
    """
    x = self._preprocess(frame_bgr)
    x.requires_grad_(False)

    logits = self.model(x)
    probs = F.softmax(logits, dim=1)[0]
    class_idx = int(torch.argmax(probs).item())
    confidence = float(probs[class_idx].item())

    self.model.zero_grad()
    score = logits[0, class_idx]
    score.backward()

    activations = self._activations[0]          # (C, H, W)
    gradients = self._gradients[0]              # (C, H, W)

    weights = gradients.mean(dim=(1, 2))         # (C,) -- global
    average pool of gradients
    cam = torch.zeros(activations.shape[1:], dtype=torch.float32)
    for c, w in enumerate(weights):
        cam += w * activations[c]
    cam = F.relu(cam)

    cam = cam.numpy()
    if cam.max() > 0:
        cam = cam / cam.max()

    cam_resized = cv2.resize(cam, (frame_bgr.shape[1], frame_bgr.
        shape[0]))
    heatmap = cv2.applyColorMap(np.uint8(255 * cam_resized), cv2.
        COLORMAP_JET)
    overlay = cv2.addWeighted(frame_bgr, 0.6, heatmap, 0.4, 0)

    return overlay, self.class_names[class_idx], confidence

```

Listing 8: app/gradcam_engine.py

12.2.8 main.py — Desktop Application

```

"""
SmartQC -- On-Device Visual Quality Inspection (Windows / Snapdragon)

Skeleton PyQt5 app. Wire your model into 'InferenceEngine.predict()'.
Designed to run two backends side by side for the CPU-vs-NPU demo:
    - "cpu" -> onnxruntime default CPUExecutionProvider
    - "npu" -> onnxruntime QNNExecutionProvider (Snapdragon NPU)

Run:
    pip install PyQt5 opencv-python onnxruntime matplotlib numpy pillow
    python main.py
"""

import sys
import time
import sqlite3
from dataclasses import dataclass
from pathlib import Path

# Import onnxruntime BEFORE cv2 -- opencv's bundled native DLLs can
# otherwise conflict with onnxruntime's DLL loading on Windows.
import onnxruntime # noqa: F401

import cv2
import numpy as np
from PyQt5.QtCore import Qt, QTimer
from PyQt5.QtGui import QImage, QPixmap
from PyQt5.QtWidgets import (
    QApplication, QMainWindow, QWidget, QLabel, QPushButton,
    QVBoxLayout, QHBoxLayout, QFileDialog, QComboBox, QListWidget,
    QListWidgetItem, QGroupBox, QSplitter, QTabWidget, QSlider,
    QTableWidget, QTableWidgetItem
)

try:
    from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as
        FigureCanvas
    from matplotlib.figure import Figure
    HAVE_MPL = True
except ImportError:
    HAVE_MPL = False

DB_PATH = Path(__file__).parent / "smartqc_history.db"
MODEL_PATH = Path(__file__).parent / "model" / "smartqc_model.onnx" #
    your exported/optimized model

```

```

CLASS_NAMES = ["good", "defective"] # edit to match your model's output
    classes
INPUT_SIZE = (224, 224) # match your model's expected input

#
-----

# Storage
#
-----

class HistoryStore:
    def __init__(self, db_path: Path = DB_PATH):
        self.conn = sqlite3.connect(str(db_path))
        self.conn.execute("""
            CREATE TABLE IF NOT EXISTS inspections (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                ts TEXT NOT NULL,
                label TEXT NOT NULL,
                confidence REAL NOT NULL,
                latency_ms REAL NOT NULL,
                backend TEXT NOT NULL
            )
        """)
        self.conn.commit()

    def add(self, label: str, confidence: float, latency_ms: float,
            backend: str):
        self.conn.execute(
            "INSERT INTO inspections (ts, label, confidence, latency_ms,
                backend) VALUES (datetime('now'), ?, ?, ?, ?)",
            (label, confidence, latency_ms, backend),
        )
        self.conn.commit()

    def recent(self, limit=50):
        cur = self.conn.execute(
            "SELECT ts, label, confidence, latency_ms, backend FROM
                inspections ORDER BY id DESC LIMIT ?",
            (limit,),
        )
        return cur.fetchall()

    def defect_rate_series(self, limit=100):

```

```

        """Returns list of (index, is_defective) for the trend chart,
        oldest first."""
        cur = self.conn.execute(
            "SELECT label FROM inspections ORDER BY id DESC LIMIT ?", (
                limit,)
        )
        rows = [r[0] for r in cur.fetchall()][::-1]
        return [1 if lbl == "defective" else 0 for lbl in rows]

#
# -----

# Inference
#
# -----

@dataclass
class InferenceResult:
    label: str
    confidence: float
    latency_ms: float
    backend: str

class InferenceEngine:
    """
    Wraps an ONNX Runtime session. Swap `backend` between "cpu" and "npu"
    to power the benchmark toggle.

    NPU path expects the QNN Execution Provider to be available in your
    onnxruntime build (onnxruntime-qnn) and the model to have been
    optimized/quantized via Qualcomm AI Hub for the Snapdragon target.
    """

    def __init__(self, model_path: Path = MODEL_PATH):
        self.model_path = model_path
        self._sessions = {} # backend -> ort.InferenceSession
        self._mock = not model_path.exists() # falls back to mock until
            you drop in a real model

    def _get_session(self, backend: str):
        if backend in self._sessions:
            return self._sessions[backend]

import onnxruntime as ort

```

```

    if backend == "npu":
        providers = ["QNNExecutionProvider", "CPUExecutionProvider"]
    else:
        providers = ["CPUExecutionProvider"]

    session = ort.InferenceSession(str(self.model_path), providers=
        providers)
    self._sessions[backend] = session
    return session

def preprocess(self, frame_bgr: np.ndarray) -> np.ndarray:
    img = cv2.resize(frame_bgr, INPUT_SIZE)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB).astype(np.float32) /
        255.0
    img = (img - np.array([0.485, 0.456, 0.406])) / np.array([0.229,
        0.224, 0.225])
    img = np.transpose(img, (2, 0, 1))[None, ...].astype(np.float32)
    return img

def predict(self, frame_bgr: np.ndarray, backend: str = "npu") ->
    InferenceResult:
    start = time.perf_counter()

    if self._mock:
        # Placeholder so the UI is demoable before the real model is
        # wired in.
        # Replace this branch's absence (i.e. drop a real model at
        # MODEL_PATH) to go live.
        probs = np.random.dirichlet(np.ones(len(CLASS_NAMES)))
    else:
        session = self._get_session(backend)
        input_name = session.get_inputs()[0].name
        x = self.preprocess(frame_bgr)
        outputs = session.run(None, {input_name: x})
        logits = outputs[0][0]
        exp = np.exp(logits - np.max(logits))
        probs = exp / exp.sum()

    latency_ms = (time.perf_counter() - start) * 1000.0
    idx = int(np.argmax(probs))
    return InferenceResult(
        label=CLASS_NAMES[idx],
        confidence=float(probs[idx]),
        latency_ms=latency_ms,
        backend=backend,
    )

```

```
#
-----

# UI
#
-----

class TrendChart(QWidget):
    def __init__(self):
        super().__init__()
        layout = QVBoxLayout(self)
        if HAVE_MPL:
            self.figure = Figure(figsize=(4, 2.5))
            self.canvas = FigureCanvas(self.figure)
            layout.addWidget(self.canvas)
        else:
            layout.addWidget(QLabel("Install matplotlib to see the defect
                                     -rate trend chart."))

    def update_chart(self, series):
        if not HAVE_MPL:
            return
        self.figure.clear()
        ax = self.figure.add_subplot(111)
        if series:
            # rolling defect rate over a small window
            window = 5
            rolling = [
                sum(series[max(0, i - window + 1): i + 1]) / min(window,
                                                                i + 1)
                for i in range(len(series))
            ]
            ax.plot(rolling, color="#000000")
            ax.set_ylim(0, 1)
            ax.set_title("Defect rate (rolling)", fontsize=9)
            ax.set_xlabel("Inspection #", fontsize=8)
        else:
            ax.text(0.5, 0.5, "No data yet", ha="center", va="center")
        self.canvas.draw()

class SmartQCWindow(QMainWindow):
    def __init__(self):
        super().__init__()
```

```
self.setWindowTitle("SmartQC -- On-Device Visual Quality
    Inspection")
self.resize(1100, 650)

self.engine = InferenceEngine()
self.store = HistoryStore()
self.cap = None
self.current_frame = None

self._build_ui()

self.timer = QTimer()
self.timer.timeout.connect(self._update_frame)

# -- UI construction -----

def _build_ui(self):
    central = QWidget()
    self.setCentralWidget(central)
    root = QHBoxLayout(central)

    splitter = QSplitter(Qt.Horizontal)
    root.addWidget(splitter)

    # Left: camera / preview
    left = QWidget()
    left_layout = QVBoxLayout(left)

    self.video_label = QLabel("Camera preview")
    self.video_label.setAlignment(Qt.AlignCenter)
    self.video_label.setMinimumSize(480, 360)
    self.video_label.setStyleSheet("background:#111; color:#888;
        border-radius:6px;")
    left_layout.addWidget(self.video_label)

    controls = QHBoxLayout()
    self.start_btn = QPushButton("Start Webcam")
    self.start_btn.clicked.connect(self._toggle_webcam)
    controls.addWidget(self.start_btn)

    self.upload_btn = QPushButton("Upload Image")
    self.upload_btn.clicked.connect(self._upload_image)
    controls.addWidget(self.upload_btn)

    self.backend_combo = QComboBox()
    self.backend_combo.addItems(["cpu (local)", "npu (AI Hub
        optimized, pending EP deploy)"])
```

```

controls.addWidget(QLabel("Backend:"))
controls.addWidget(self.backend_combo)

self.inspect_btn = QPushButton("Run Inspection")
self.inspect_btn.clicked.connect(self._run_inspection)
controls.addWidget(self.inspect_btn)

left_layout.addLayout(controls)

# -- Confidence threshold control
-----

threshold_row = QHBoxLayout()
threshold_row.addWidget(QLabel("Confidence threshold:"))
self.threshold_slider = QSlider(Qt.Horizontal)
self.threshold_slider.setMinimum(50)
self.threshold_slider.setMaximum(99)
self.threshold_slider.setValue(70)
self.threshold_slider.setTickPosition(QSlider.TicksBelow)
self.threshold_slider.setTickInterval(5)
self.threshold_slider.valueChanged.connect(self._update_threshold_label)
threshold_row.addWidget(self.threshold_slider)

self.threshold_value_label = QLabel("70%")
self.threshold_value_label.setMinimumWidth(40)
threshold_row.addWidget(self.threshold_value_label)
left_layout.addLayout(threshold_row)

self.threshold_hint = QLabel(
    "Predictions below this confidence are flagged UNCERTAIN  
instead of forced good/bad."
)
self.threshold_hint.setStyleSheet("color:#888; font-size:11px;")
self.threshold_hint.setWordWrap(True)
left_layout.addWidget(self.threshold_hint)

self.result_label = QLabel("Result: --")
self.result_label.setStyleSheet("font-size:16px; font-weight:bold  
; padding:8px;")
left_layout.addWidget(self.result_label)

self.latency_label = QLabel("Latency: --")
left_layout.addWidget(self.latency_label)

self.benchmark_btn = QPushButton("Run Timing Comparison (local  
CPU)")
self.benchmark_btn.clicked.connect(self._run_benchmark)

```

```
left_layout.addWidget(self.benchmark_btn)

self.benchmark_note = QLabel(
    "Note: this compares repeated local inference calls, not a\n"
    "validated NPU benchmark.\n"
    "The model is compiled/optimized for the Snapdragon NPU via\n"
    "Qualcomm AI Hub\n"
    "(see the AI Hub job link in the README) -- NPU execution\n"
    "provider deployment\n"
    "is the final integration step."
)
self.benchmark_note.setStyleSheet("color:#888; font-size:11px;")
left_layout.addWidget(self.benchmark_note)

self.benchmark_label = QLabel("")
left_layout.addWidget(self.benchmark_label)

splitter.addWidget(left)

# Right: tabs for history + trend
right = QTabWidget()

history_tab = QWidget()
history_layout = QVBoxLayout(history_tab)
self.history_list = QListWidget()
history_layout.addWidget(self.history_list)
right.addTab(history_tab, "History")

trend_tab = QWidget()
trend_layout = QVBoxLayout(trend_tab)
self.trend_chart = TrendChart()
trend_layout.addWidget(self.trend_chart)
right.addTab(trend_tab, "Trend")

explain_tab = QWidget()
explain_layout = QVBoxLayout(explain_tab)
self.explain_label = QLabel(
    "Run 'Show Explainability Heatmap' below to see which region\n"
    "n"
    "of the image most influenced the model's decision."
)
self.explain_label.setAlignment(Qt.AlignCenter)
self.explain_label.setWordWrap(True)
self.explain_label.setMinimumSize(400, 320)
self.explain_label.setStyleSheet("background:#111; color:#888;\n"
    "border-radius:6px;")
explain_layout.addWidget(self.explain_label)
```

```
self.explain_btn = QPushButton("Show Explainability Heatmap (Grad
-CAM) ")
self.explain_btn.clicked.connect(self._run_gradcam)
explain_layout.addWidget(self.explain_btn)

self.explain_status = QLabel("")
self.explain_status.setWordWrap(True)
self.explain_status.setStyleSheet("color:#888; font-size:11px;")
explain_layout.addWidget(self.explain_status)

right.addTab(explain_tab, "Explainability")

batch_tab = QWidget()
batch_layout = QVBoxLayout(batch_tab)

batch_controls = QHBoxLayout()
self.batch_folder_btn = QPushButton("Select Folder to Batch
Inspect")
self.batch_folder_btn.clicked.connect(self._run_batch_inspection)
batch_controls.addWidget(self.batch_folder_btn)
batch_layout.addLayout(batch_controls)

self.batch_summary_label = QLabel("No batch run yet.")
self.batch_summary_label.setStyleSheet("font-weight:bold; padding
:4px;")
batch_layout.addWidget(self.batch_summary_label)

self.batch_table = QTableWidgetItem()
self.batch_table.setColumnCount(4)
self.batch_table.setHorizontalHeaderLabels(["File", "Result", "
Confidence", "Latency (ms)"])
self.batch_table.horizontalHeader().setStretchLastSection(True)
batch_layout.addWidget(self.batch_table)

self.batch_export_btn = QPushButton("Export Results to CSV")
self.batch_export_btn.clicked.connect(self._export_batch_csv)
self.batch_export_btn.setEnabled(False)
batch_layout.addWidget(self.batch_export_btn)

right.addTab(batch_tab, "Batch Inspect")

splitter.addWidget(right)
splitter.setSizes([650, 450])

self._refresh_history()
self._gradcam = None # lazy-loaded on first use
```

```
self._batch_results = [] # populated by _run_batch_inspection

# -- Webcam
-----

def _toggle_webcam(self):
    if self.cap is None:
        self.cap = cv2.VideoCapture(0)
        self.timer.start(30)
        self.start_btn.setText("Stop Webcam")
    else:
        self.timer.stop()
        self.cap.release()
        self.cap = None
        self.start_btn.setText("Start Webcam")

def _update_frame(self):
    if self.cap is None:
        return
    ok, frame = self.cap.read()
    if not ok:
        return
    self.current_frame = frame
    self._display_frame(frame)

def _display_frame(self, frame_bgr):
    rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
    h, w, ch = rgb.shape
    qimg = QImage(rgb.data, w, h, ch * w, QImage.Format_RGB888)
    pix = QPixmap.fromImage(qimg).scaled(
        self.video_label.width(), self.video_label.height(), Qt.
        KeepAspectRatio
    )
    self.video_label.setPixmap(pix)

def _upload_image(self):
    path, _ = QFileDialog.getOpenFileName(self, "Select image", "", "
        Images (*.png *.jpg *.jpeg)")
    if not path:
        return
    frame = cv2.imread(path)
    if frame is None:
        return
    self.current_frame = frame
    self._display_frame(frame)
```

```
# -- Inspection
-----

def _update_threshold_label(self, value):
    self.threshold_value_label.setText(f"{value}%")

def _run_inspection(self):
    if self.current_frame is None:
        self.result_label.setText("Result: no frame captured yet")
        return

    # both dropdown options currently execute on local CPU -- the AI
    # Hub
    # optimized model is compiled and validated for the Snapdragon
    # NPU,
    # but the QNN execution provider is not yet wired up in this
    # build.
    result = self.engine.predict(self.current_frame, backend="cpu")
    self._apply_result(result)
    self.store.add(result.label, result.confidence, result.latency_ms
        , result.backend)
    self._refresh_history()

def _apply_result(self, result: InferenceResult):
    threshold = self.threshold_slider.value() / 100.0
    if result.confidence < threshold:
        color = "#000000"
        display_text = f"Result: UNCERTAIN (best guess: {result.
            label.upper()}, {result.confidence*100:.1f}%)"
    else:
        color = "#000000"
        display_text = f"Result: {result.label.upper()} ({result.
            confidence*100:.1f}%)"
    self.result_label.setText(display_text)
    self.result_label.setStyleSheet(f"font-size:16px; font-weight:
        bold; padding:8px; color:{color};")
    self.latency_label.setText(f"Latency: {result.latency_ms:.1f} ms
        [{result.backend.upper()}]")

def _run_benchmark(self):
    if self.current_frame is None:
        self.benchmark_label.setText("Capture or upload a frame first
            .")
        return
    run1 = self.engine.predict(self.current_frame, backend="cpu")
    run2 = self.engine.predict(self.current_frame, backend="cpu")
    self.benchmark_label.setText(
```

```

        f"Run 1: {run1.latency_ms:.1f} ms    |    Run 2: {run2.
            latency_ms:.1f} ms    "
        f"(both local CPU -- see note above)"
    )

def _run_gradcam(self):
    if self.current_frame is None:
        self.explain_status.setText("Capture or upload a frame first.
            ")
        return

    self.explain_status.setText("Computing heatmap...")
    QApplication.processEvents()

    try:
        if self._gradcam is None:
            from gradcam_engine import GradCAM
            self._gradcam = GradCAM()

        overlay, label, confidence = self._gradcam.generate(self.
            current_frame)

        rgb = cv2.cvtColor(overlay, cv2.COLOR_BGR2RGB)
        h, w, ch = rgb.shape
        qimg = QImage(rgb.data, w, h, ch * w, QImage.Format_RGB888)
        pix = QPixmap.fromImage(qimg).scaled(
            self.explain_label.width(), self.explain_label.height(),
            Qt.KeepAspectRatio
        )
        self.explain_label.setPixmap(pix)
        clean_label = "good" if "good" in label.lower() or "fresh" in
            label.lower() else "defective"
        self.explain_status.setText(
            f"Heatmap for prediction: {clean_label.upper()} ({
                confidence*100:.1f}%). "
            f"Warmer colors (red/yellow) show the regions that most
                influenced this result."
        )
    except FileNotFoundError:
        self.explain_status.setText(
            "Model weights not found at model/smartqc_weights.pt --
                run train.py first."
        )
    except Exception as e:
        self.explain_status.setText(f"Could not generate heatmap: {e}
            ")

```

```
# -- Batch inspection -----

def _run_batch_inspection(self):
    folder = QFileDialog.getExistingDirectory(self, "Select folder of
        images to inspect")
    if not folder:
        return

    folder_path = Path(folder)
    image_exts = {".png", ".jpg", ".jpeg", ".bmp"}
    image_paths = sorted(
        p for p in folder_path.iterdir() if p.suffix.lower() in
            image_exts
    )

    if not image_paths:
        self.batch_summary_label.setText("No images found in that
            folder.")
        return

    threshold = self.threshold_slider.value() / 100.0
    results = []
    good_count = defective_count = uncertain_count = 0

    for path in image_paths:
        frame = cv2.imread(str(path))
        if frame is None:
            continue
        result = self.engine.predict(frame, backend="cpu")
        if result.confidence < threshold:
            display_label = "UNCERTAIN"
            uncertain_count += 1
        elif result.label == "good":
            display_label = "GOOD"
            good_count += 1
        else:
            display_label = "DEFECTIVE"
            defective_count += 1
        results.append((path.name, display_label, result.confidence,
            result.latency_ms))

    self._batch_results = results
    self.batch_summary_label.setText(
        f"{len(results)} images inspected -- {good_count} good, "
        f"{defective_count} defective, {uncertain_count} uncertain."
    )
```

```

self.batch_table.setRowCount(len(results))
for row, (name, label, conf, latency) in enumerate(results):
    self.batch_table.setItem(row, 0, QTableWidgetItem(name))
    self.batch_table.setItem(row, 1, QTableWidgetItem(label))
    self.batch_table.setItem(row, 2, QTableWidgetItem(f"{conf
        *100:.1f}%"))
    self.batch_table.setItem(row, 3, QTableWidgetItem(f"{latency
        :.1f}"))

self.batch_export_btn.setEnabled(bool(results))

def _export_batch_csv(self):
    if not self._batch_results:
        return
    save_path, _ = QFileDialog.getSaveFileName(
        self, "Save batch results as CSV", "smartqc_batch_results.csv",
        "CSV Files (*.csv)"
    )
    if not save_path:
        return
    import csv
    with open(save_path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["file", "result", "confidence", "latency_ms"
            ])
        for name, label, conf, latency in self._batch_results:
            writer.writerow([name, label, f"{conf*100:.1f}%", f"{
                latency:.1f}"])
    self.batch_summary_label.setText(
        self.batch_summary_label.text() + f" (saved to {Path(
            save_path).name})"
    )

# -- History / trend -----

def _refresh_history(self):
    self.history_list.clear()
    for ts, label, conf, latency, backend in self.store.recent():
        item = QListWidgetItem(f"{ts} | {label} ({conf*100:.1f}%)
            | {latency:.1f} ms | {backend}")
        self.history_list.addItem(item)
    self.trend_chart.update_chart(self.store.defect_rate_series())

def main():
    app = QApplication(sys.argv)
    window = SmartQCWindow()

```

```
    window.show()
    sys.exit(app.exec_())

if __name__ == "__main__":
    main()
```

Listing 9: app/main.py

Appendix C: Repository and Assets

- **Demo:** [Link](#)
- **GitHub:** [Link](#)
- **LinkedIn:** [Link](#)
- **Qualcomm AI Hub compile job:** jpe782z15, target device: Snapdragon X Elite CRD